

POSTGRES ON OPENSIFT WORKSHOP



Stephen Hall - Senior Sales Engineer, EDB
Soren Boss - Strategic Account Executive, EDB

2th of June, 2026



Agenda

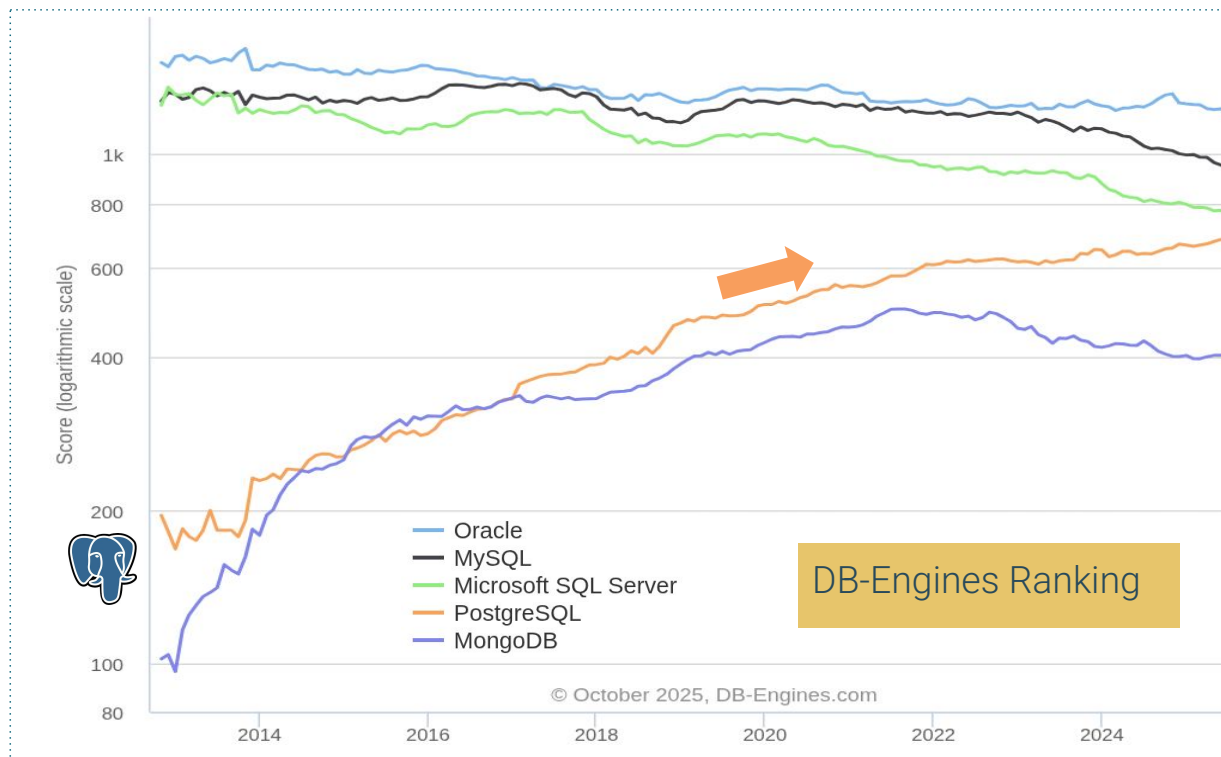
Commencer	Fin	Session
14:00	14:10	Intro to Postgres, EDB and CNPG
14:10	14:30	Architecture
14:30	15:00	Workshop : 4 exercises
15:00	15:55	Workshop : 5 exercises
15:55	16:00	Wrap up and goodbye!



Introduction to Postgres and EDB



PostgreSQL is Booming !

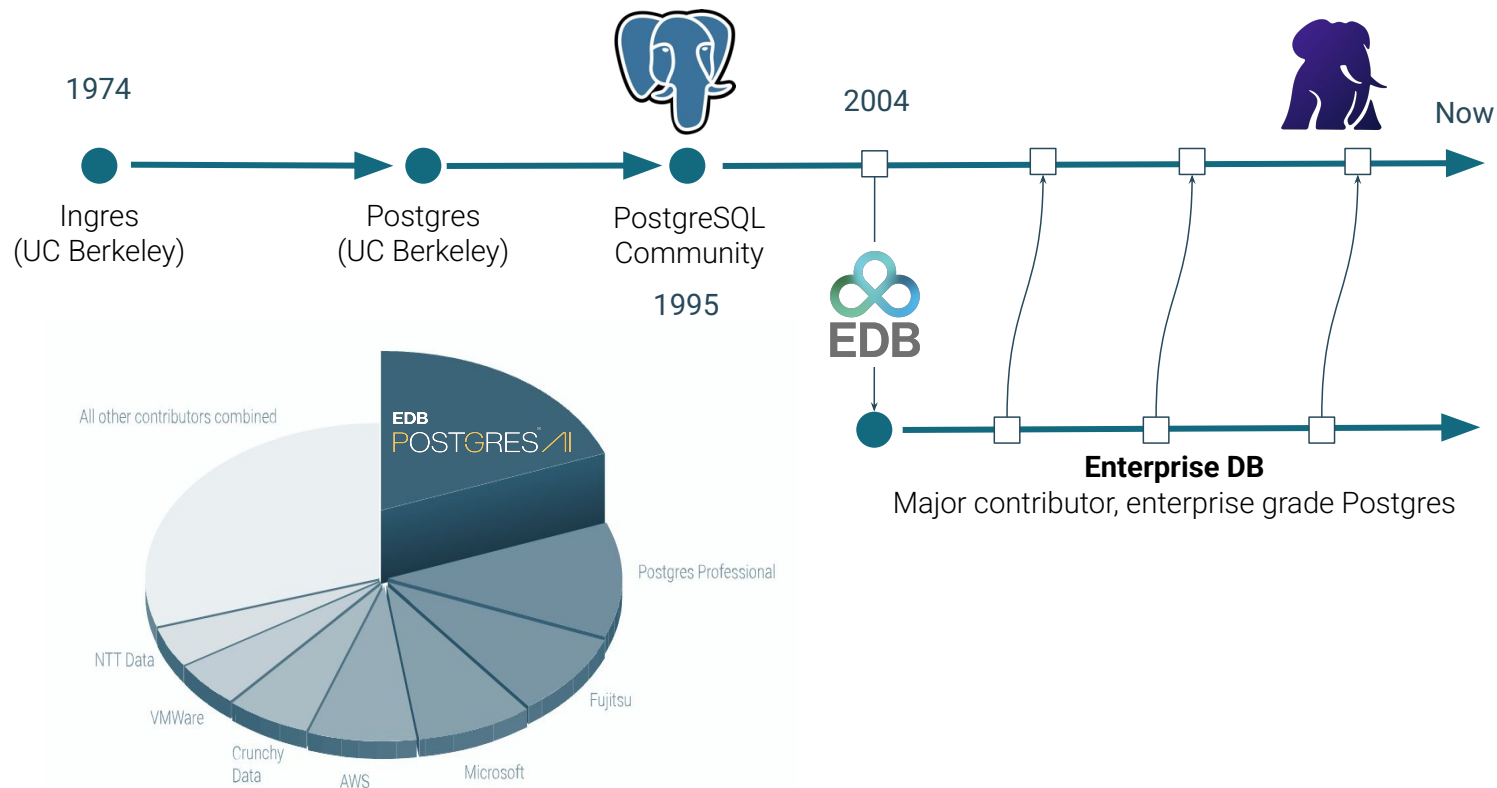


35% 

Thirty-five percent of Enterprises will consider Postgres for their next project



EnterpriseDB Biggest PostgreSQL Contributor



CLIENTS

AGENTS

APPLICATIONS

USERS

**EDB POSTGRES[®]
AI**

WORKLOADS

OPERATIONAL

RELATIONAL

DOCUMENT

VECTOR

GRAPH

OPERATIONAL

ANALYTICAL

WAREHOUSE

LAKEHOUSE

REAL-TIME

BATCH

AGENTIC

RUNTIME

BUILDING

SEMANTIC SEARCH

RAG

MACHINE LEARNING

PLATFORM SERVICES

GOVERNANCE

SECURITY

DATA INTEGRATION

MIGRATIONS

OBSERVABILITY

LIFECYCLE

HA/DR

DATA LAYER

LAKEHOUSE
OBJECT STORAGE

DATABASES
POSTGRESQL

STREAMS
EVENT DATA

DEPLOYMENT OPTIONS

IBM MAINFRAME

SOVEREIGN DC

PUBLIC CLOUD

NVIDIA GPU

EDB AI FACTORY HW





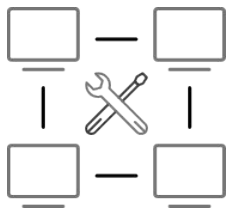
CNPG Operator

Postgres on Kubernetes



Challenges running Postgres in Kubernetes

Kubernetes accelerates the future — but also introduces complexity that can slow you down.



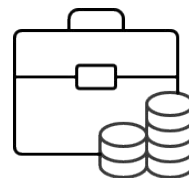
- Deployment complexity

- Infrastructure teams manage many different systems at scale. Introducing a new database only adds to the complexity.
- Platform engineers need solutions that are Day 2-ready, with cloud-native tooling and easy CI/CD integration.



- Operational overhead

- Smooth Day 2 ops requires knowledge and expertise of both container and database technology.
- Consistent operations can be hampered by human error and lack of expertise.



- Costly downtime

- Mission-critical apps require data that is consistent, compliant, and complete — without sacrificing performance.
- Distributed DBs are a good HA solution, but introduce operational complexity.



EDB Postgres on Kubernetes

EDB delivers continuous high availability for the highest confidence in your data-intensive workloads.



▪ Cloud-native automation

- Automate database administration across regions and distributed cluster operations.
- Deploy, manage, monitor, and streamline using declarative configuration with the Kubernetes API.



▪ Consistent operations

- Implement best practices by relying on EDB's expertise in Postgres and Kubernetes to stay up to date on Cloud Native best practices
- Eliminate human error for consistent deployment of data services.



▪ Self-healing architecture

- Automate recovery and rejoin of failed Postgres nodes, removing operational overhead.
- Quickly restore functionality with failover, switchover, backup, recovery, and rolling upgrades.



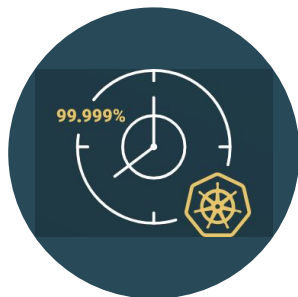
EDB CloudNativePG

DEPLOYMENT, AUTOMATION, AND SMOOTH OPERATIONS FOR POSTGRES ON KUBERNETES



CloudNativePG

- Built by EDB and contributed to the CNCF
- Most popular Operator for PostgreSQL on Kubernetes
- Automate the deployment and management of Postgres in Kubernetes
- Commercial support available from EDB experts



EDB CloudNativePG Cluster

- Active/Passive architecture for high-availability
- Security features like TDE, Oracle Compatibility support and, LTS
- Deploy and manage Postgres with automation and reliability for cloud-native infrastructure needs.



EDB CloudNativePG Global Cluster

- Active/Active Geo-distributed architecture
- Self-healing clusters
- Up to 99.999% uptime and 5x throughput



Feature overview



- Deployment flexibility

- Ready to deploy in your preferred cloud-native environment. Run on premises, in public cloud providers, and as a managed cloud service. Fully supported for Red Hat OpenShift, IBM Power, and z/Linux.



- Monitoring and observability

- Get enhanced diagnostics using EDB Postgres AI to empower strategic decision-making, helping ensure optimal performance and uptime. Take advantage of the built-in Prometheus exporter and native integration with infrastructure log management.



- Enterprise-grade security

- Deploy any application with confidence, using security technologies like Transparent Data Encryption, audit trail, privilege analysis, and firewall support.



Kubernetes timeline

- 2014, June: Google open sources Kubernetes
- 2015, July: Version 1.0 is released
- 2015, July: Google and Linux Foundation start the CNCF
- 2016, November: The operator pattern is introduced in a blog post
- 2018, August: The Community takes the lead
- 2019, April: Version 1.14 introduces **Local Persistent Volumes**
- 2019, August: EDB team starts the Kubernetes initiative
- 2020, June: we publish this blog about benchmarking local PVs on bare metal
- 2020, June: Data on Kubernetes Community founded
- 2021, February: EDB Cloud Native Postgres (CNP) 1.0 released
- 2022, May: **EDB donates CNP** and open sources it under CloudNativePG
- 2025, January: CloudNativePG was recognized as an official **#CNCF** project



A kubernetes operator for Postgres



Kubernetes adoption is rising and it is already the de facto **standard orchestration tool**



PostgreSQL clusters “**management the kubernetes way**” enables many cloud native usage patterns, e.g. spinning up, disposable clusters during tests, one cluster per microservice and one database per cluster



CNPG tries to encode years of experience managing PostgreSQL clusters into **an Operator which should automate all the known tasks a user could be willing to do**

Our PostgreSQL operator must simulate a part of the work of a DBA



Autopilot

It automates the steps that a human operator would do to deploy and to manage a Postgres database inside Kubernetes, including automated failover.



Security

A man in a tactical vest with the word "SECURITY" printed on the back, holding a walkie-talkie to his mouth. The image is faded and serves as a background for the slide.

SECURITY

CloudNativePG is secured by default.



It doesn't rely on statefulsets and uses its own way to manage persistent volume claims where the PGDATA is stored.

Data persistence



Designed for Kubernetes

It's entirely declarative, and directly integrates with the Kubernetes API server to update the state of the cluster — for this reason, it does not require an external failover management tool.



Features

Deployment	Administration	Backup & Recovery	Monitoring	Security	High Availability
Kubernetes operator	Single node	Backup	Prometheus	TDE	Switchover
Kubernetes plugin	Cluster (Multi node)	Recovery	Grafana dashboards	Certificates	Failover
EDB Postgres (EPAS)	PostgreSQL configuration	PITR	Postgres Enterprise Manager	Data redaction	Scale out / scale down
PostGIS	Pooling	Volume Snapshots	Logging	Password management	Minor / Major updates



What CNPG gives you : Day 1 and Day 2

CloudNativePG automates what a DBA would do; deploy, manage, heal, back up, upgrade; all through Kubernetes-native YAML.

Day 1 — get it running

Operator + OLM plugin

Install from OperatorHub in one click

Single-node or HA cluster

Declare instances: 1 or instances: 3 in YAML

PostgreSQL config as code

All pg parameters live in the Cluster manifest

EPAS, PostGIS, pgvector

Swap image tag to change Postgres flavour

TLS certificates out of the box

Mutual TLS between all cluster components

Day 2 — keep it healthy

Auto failover in < 30 seconds

Lease-based fencing prevents split-brain

Backup + PITR to object storage

Barman Cloud to Minio, S3, GCS, or Azure

Prometheus + Grafana built-in

~200 metrics, no extra exporter to install

Minor + major version upgrades

Change image tag — operator handles pg_upgrade

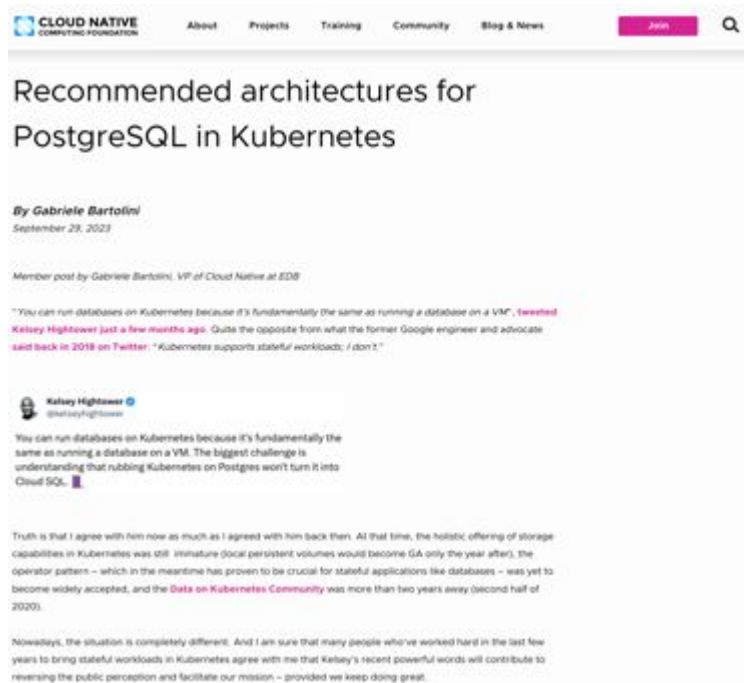
PgBouncer connection pooling

Declare a Pooler CRD — operator manages it



Recommended architectures

<https://www.cncf.io/blog/2023/09/29/recommended-architectures-for-postgresql-in-kubernetes/>



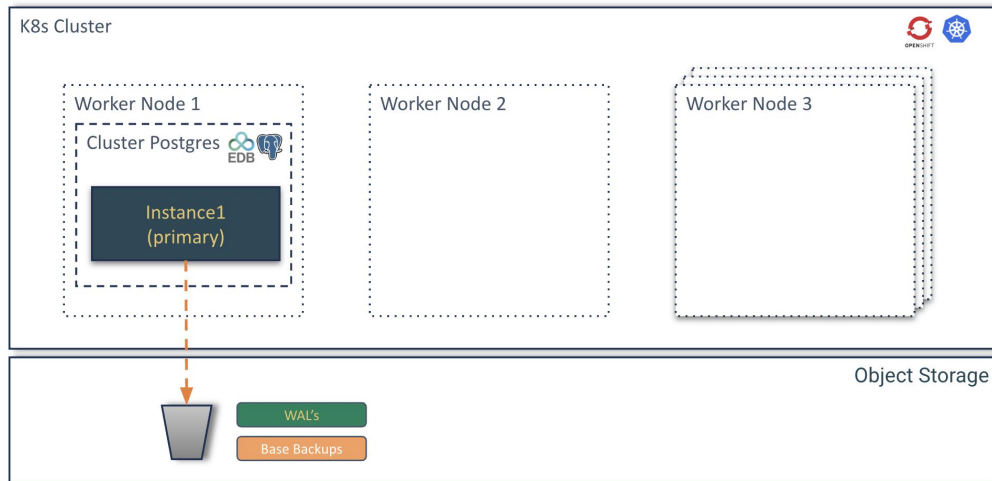
Use Cases



Use case 1 architecture

A single database is the simplest setup, involving one instance of a database server.

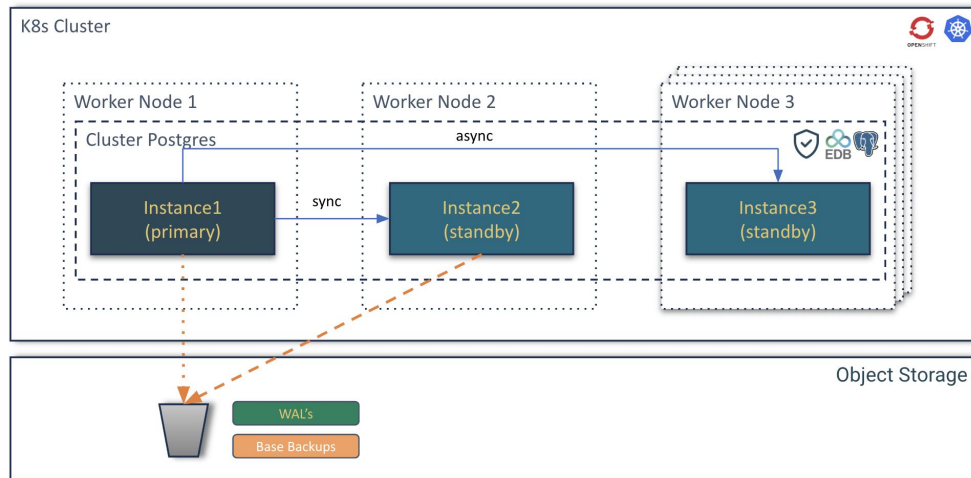
- Development and testing environments
- Small applications with low traffic
- Non-critical data analysis
- Applications with high tolerance for downtime
- Cost-sensitive projects



Use case 2 architecture

An HA database setup aims to minimize downtime by having redundant components. If one component fails, another takes over automatically or with minimal intervention. This usually involves techniques like clustering, replication, or mirroring **within the same data center or availability zone**.

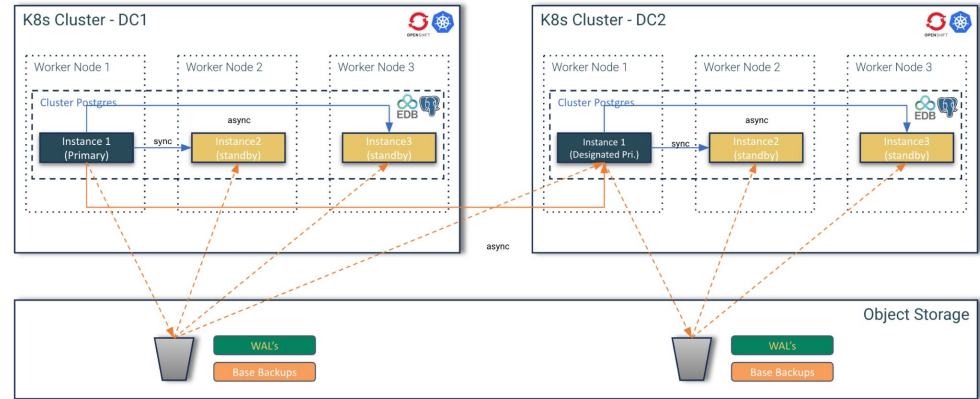
- Business critical Applications
- Applications with stringent SLAs
- Real-time systems
- Improving user experience
- Minimizing planned downtime



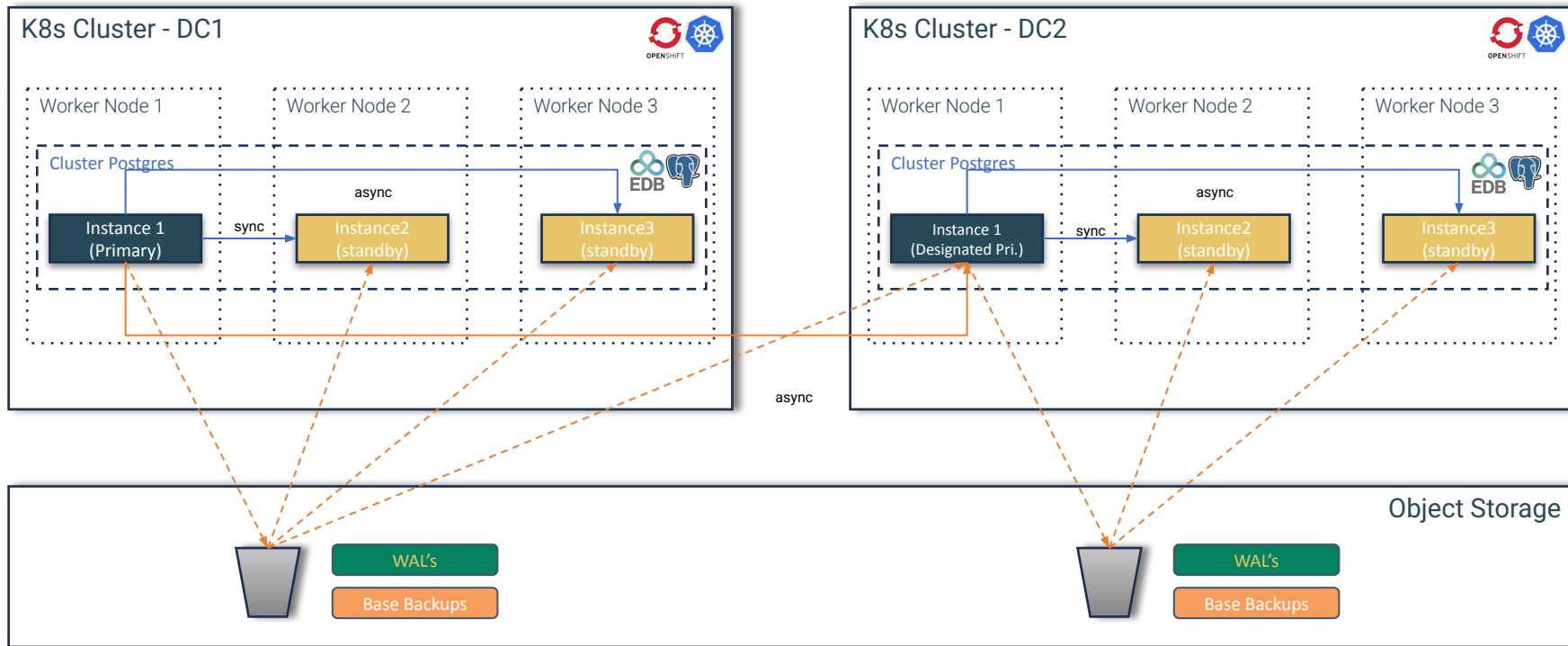
Use case 3 architecture

A DR database setup focuses on protecting data and ensuring business continuity in the event of a large-scale disaster affecting an entire data center or region (e.g., natural disasters, power outages, cyberattacks). This typically involves replicating data to a geographically separate location.

- Regulatory compliance
- Protecting against catastrophic data loss
- Ensuring business continuity for mission-critical systems



Use case 3 architecture



The Workshop



What you will do today

- Kubernetes plugin install
- Check the CloudNativePG operator status
- Postgres cluster install
- Insert data in the cluster
- Failover
- Backup
- Recovery
- Scale out/down
- Fencing
- Hibernation

What has been done

- CNPG operator installation
- Kubernetes plugin install **kubect1-cnp**
- S3 bucket creation

Deployment

Administration

Backup and
Recovery

High Availability

Monitoring

Last CloudNativePG tested version is 1.25



Workshop architecture

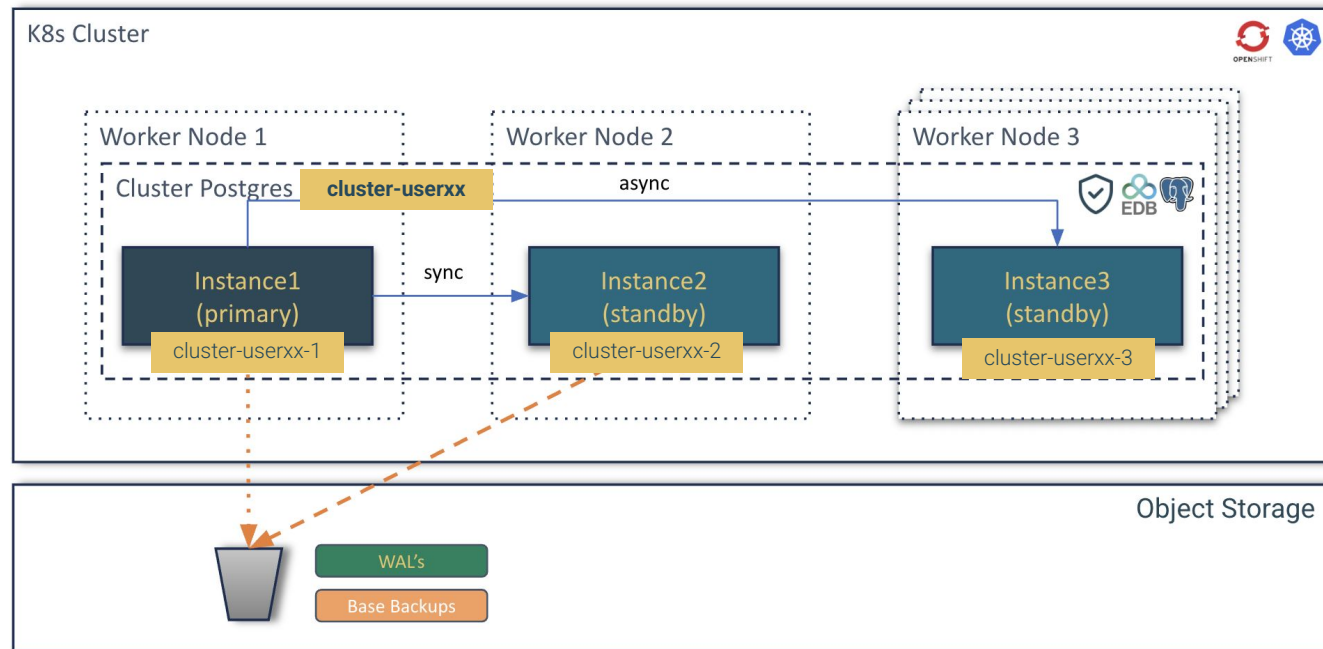
Name space's PG cluster edb-emea-userXX

Services

- cluster-userxx-rw
- cluster-userxx-ro
- cluster-userxx-rw

Name space's S3 storage minio

minio



How the project is build

You will run script to do all the task:

- 03_check_operator.sh
- 04_prepare_cluster.sh
- 05_install_cluster.sh
- ...

Variable are stored in

- config.sh

```
EXPLORER
(WORKSPACE)
  workshop
    > yaml
    $ .credentials.sh
    $ 01_install_plugin.sh
    $ 02_install_operator.sh
    $ 03_check_operator_installed.sh
    $ 04_prepare_cluster.sh
    $ 05_install_cluster.sh
    $ 06_show_status.sh
    $ 07_insert_data.sh
    $ 08_promote.sh
    $ 09_upgrade.sh
    $ 10_backup_cluster.sh
    $ 11_backup_describe.sh
    $ 12_restore_cluster.sh
    $ 13_failover.sh
    $ 14_scale_out.sh
    $ 15_scale_down.sh
    $ 16_pitr_insert_two_lines.sh
    $ 17_pitr_backup.sh
    $ 18_pitr_insert_new_line.sh
    $ 19_pitr_restore_line_one.sh
    $ 20_upgrade_major_version.sh
    $ 30_fencing_on.sh
    $ 31_fencing_off.sh
    $ 32_hibernation_on.sh
    $ 33_hibernation_off.sh
    $ 60_pgouncer.sh
    $ 61_pgouncer_port_forward.sh
    $ 99_delete_env.sh
    $ aws_secrets.sh
    $ commands.sh
    $ config.sh
```

Custom Ressources
are in YAML folder

They are call by
scripts .sh



Example of a script

```
kubectl-cnp apply -n edb-ema-e-userxx -f ./yaml/cluster-sample.yaml
```

05_install_cluster.sh

```
#!/bin/bash
. ./config.sh
. ./env.sh

#Doc
echo "05" > ./docs/docid

if [[ $? -eq 1 ]]; then
    echo "Error occurred. Please fix the problem."
    exit 1
fi

# Create namespace if does not exists
. ./create_namespace.sh

print_command "${kubectl_cmd} apply -n ${namespace}
-f ./yaml/cluster-sample.yaml\n"

envsubst < ./yaml/cluster-sample.yaml |
${kubectl_cmd} apply -n ${namespace} -f-
```

config.sh

```
# Kubernetes environment configuration
export namespace="edb-${region}-${id}"
export kubectl_cmd="oc"
export kubectl_cnp="kubectl-cnp"
...
```

cluster-sample.yaml

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: $cluster_name
spec:
  instances: $postgres_instances
  logLevel: info
  imageName: "$postgres_default_image"
  imagePullSecrets:
  ...
```



Interactive session
It's time to go hands-on!



Hand-on documentation



Download this presentation

<http://bit.ly/4dEQwqc>

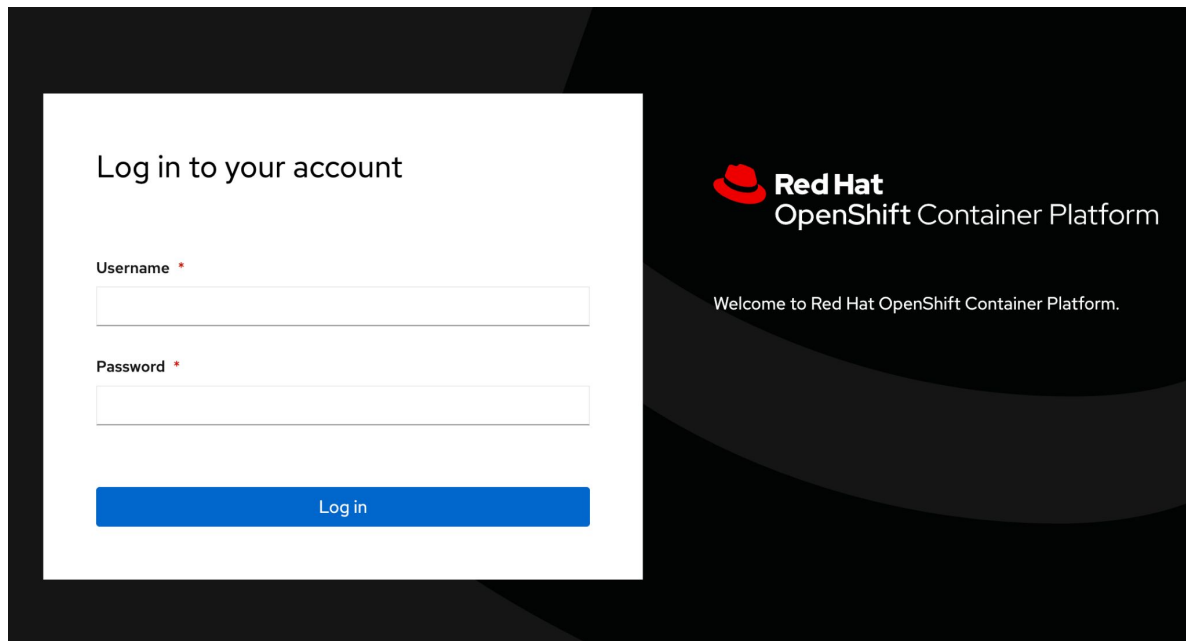


Environment

OpenShift Console	https://console-openshift-console.apps.ocp.t8s7c.sandbox5452.opentlc.com
OpenShift User name	user01..user20
OpenShift Password	EDB-RH-PASS26
Devspace url:	https://devspaces.apps.ocp.t8s7c.sandbox5452.opentlc.com/
Minio UI	https://minio-ui-minio.apps.ocp.t8s7c.sandbox5452.opentlc.com
Minio API	https://minio-api-minio.apps.ocp.t8s7c.sandbox5452.opentlc.com
Minio User	minio
Minio password	edb-workshop
Github create OpenShit env	https://github.com/michael-bang/devspace-edb
Github create Postgres demo	https://github.com/sergioenterprisedb/edb-postgres-for-kubernetes-in-openshift/tree/2026-aarhus



Red Hat OpenShift Login



The image shows a login interface for the Red Hat OpenShift Container Platform. It features a dark background with a white login form on the left and a dark sidebar on the right. The form contains fields for Username and Password, and a Log in button. The sidebar displays the Red Hat logo and the text 'Red Hat OpenShift Container Platform' and 'Welcome to Red Hat OpenShift Container Platform.'

Log in to your account

Username *

Password *

Log in

 **Red Hat**
OpenShift Container Platform

Welcome to Red Hat OpenShift Container Platform.

Red Hat OpenShift environment

The screenshot shows the Red Hat OpenShift console interface. On the left is a sidebar with navigation links: Home, Favorites, Ecosystem, Helm, Workloads, Networking, Storage, Builds, Observe, Compute, User Management, and Administration. The main content area displays the 'Installed Operators' for the 'openshift-image-registry' project. A blue banner at the top of the main area states: 'You are logged in as a temporary administrative user. Update the [cluster OAuth configuration](#) to allow others to log in.' Below this, the 'Installed Operators' section includes a search bar and a table of installed operators. The table has columns for Name, Managed Namespaces, Status, and Provided APIs. One operator is listed: 'EDB Postgres for Kubernetes', version 1.28.0, provided by EDB. Its status is 'Succeeded' and 'Up to date'. The 'Managed Namespaces' column shows 'NS openshift-image-registry' and a note that the operator is running in 'openshift-operators' but is managing this namespace. The 'Provided APIs' column lists links for Backups, Cluster Image Catalog, Cluster, Postgres Database, and a 'View 6 more...' link.

Red Hat OpenShift

You are logged in as a temporary administrative user. Update the [cluster OAuth configuration](#) to allow others to log in.

Project: openshift-image-registry

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name Search by name...

Name	Managed Namespaces	Status	Provided APIs
EDB Postgres for Kubernetes 1.28.0 provided by EDB	openshift-image-registry The operator is running in openshift-operators but is managing this namespace	✓ Succeeded Up to date	Backups Cluster Image Catalog Cluster Postgres Database View 6 more...



Red Hat OpenShift operator

The screenshot displays the Red Hat OpenShift console interface. On the left is a sidebar with navigation links: Home, Favorites, Ecosystem, Software Catalog, Installed Operators, Helm, Workloads, Networking, Storage, Builds, Observe, Compute, User Management, and Administration. The main content area shows the details for the 'EDB Postgres for Kubernetes' operator, version 128.0, provided by EDB. A top banner indicates the user is logged in as a temporary administrative user and suggests updating the cluster OAuth configuration. Below this, a breadcrumb trail shows 'Installed Operators' > 'Operator details'. The operator's name 'EDB Postgres for Kubernetes' and version '128.0 provided by EDB' are displayed. A star icon and an 'Actions' dropdown menu are also present. A horizontal tab bar includes links for Details (selected), YAML, Subscription, Events, All instances, Backups, Cluster Image Catalog, Cluster, Postgres Database, Failover Quorum, and Image Catalog. The 'Provided APIs' section contains nine cards, each representing a different API: Backups, Cluster Image Catalog, Cluster, Postgres Database, Failover Quorum, Image Catalog, Pooler, Postgres Publication, and Scheduled Backups. Each card includes a brief description and a 'Create instance' link. On the right side, the 'Provider' is listed as EDB, the 'Created at' timestamp is Dec 17, 2025, 10:38 AM, and there are links for documentation and maintainers. The maintainers listed are Jonathan Gonzalez V., Jonathan Battisto, and Nicolo Fei.

You are logged in as a temporary administrative user. Update the [cluster OAuth configuration](#) to allow others to log in.

Project: openshift-image-registry

[Installed Operators](#) > Operator details

EDB Postgres for Kubernetes
128.0 provided by EDB

< Details **YAML** Subscription Events All instances Backups Cluster Image Catalog Cluster Postgres Database Failover Quorum Image Catalog >

Provided APIs

B Backups
PostgreSQL backup (physical base backup)
[Create instance](#)

CI Cluster Image Catalog
A cluster-wide catalog of PostgreSQL operand images
[Create instance](#)

C Cluster
PostgreSQL cluster (primary/standby architecture)
[Create instance](#)

PD Postgres Database
Declarative creation and management of a database on a Cluster
[Create instance](#)

FQ Failover Quorum
FailoverQuorum contains the information about the current failover quorum status of a PG cluster
[Create instance](#)

IC Image Catalog
A catalog of PostgreSQL operand images
[Create instance](#)

P Pooler
Pooler for a Postgres Cluster (with PgBouncer)
[Create instance](#)

PP Postgres Publication
Declarative creation and management of a Logical Replication Publication in a PostgreSQL Cluster
[Create instance](#)

SB Scheduled Backups
Backup scheduler for a given Postgres cluster
[Create instance](#)

Provider
EDB

Created at
Dec 17, 2025, 10:38 AM

Links
EDB Postgres for Kubernetes
<https://www.enterprisedb.com/products/postgres-ql-on-kubernetes-ha-clusters-k8s-containers-scalable>
Documentation
<https://www.enterprisedb.com/docs/postgres-for-kubernetes/latest/>

Maintainers
Jonathan Gonzalez V. jonathan.gonzalez@enterprisedb.com
Jonathan Battisto jonathan.battisto@enterprisedb.com
Nicolo Fei nicolo.fei@enterprisedb.com
Gabriele Bartolini gabriele.bartolini@enterprisedb.com

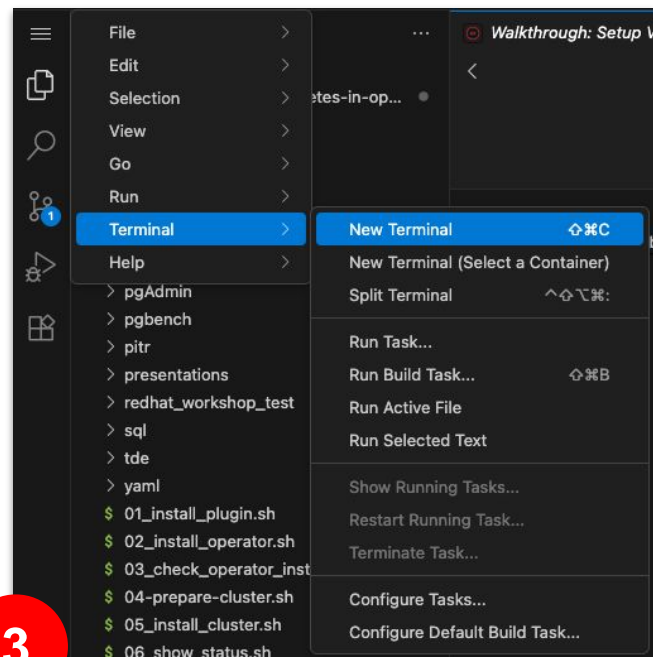
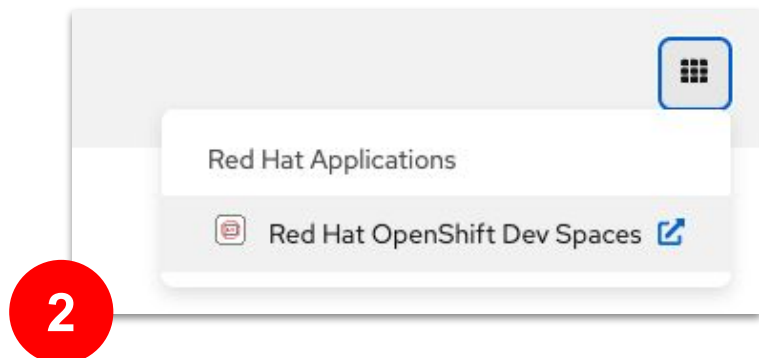
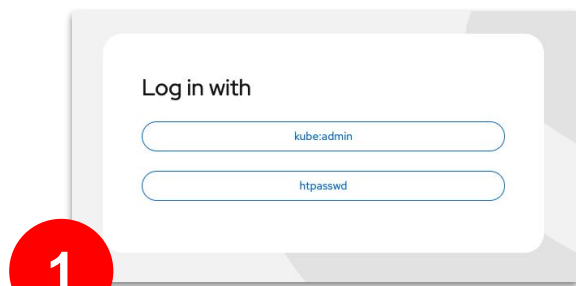


Hands-on



OpenShift and DevSpaces access

<https://github.com/raphael-chir/edb-postgres-for-kubernetes-in-openshift>



Plugging and OpenShift login

Scripts

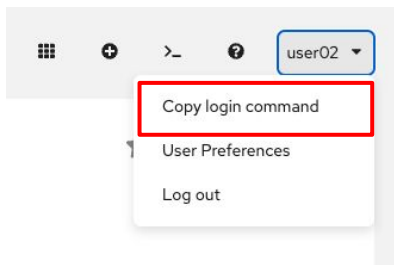
01_install_plugin.sh

Install plugin

CloudNativePG provides a plugin for kubectl to manage a Postgres cluster in Kubernetes. You can manage status, promote, certificates, restart, reload, maintenance, report, logs, destroy, hibernation, backups, ...

Doc [link](#)

OpenShift Login and namespace creatop,



Code

```
# Install plugin

curl -sSfL \
  https://github.com/EnterpriseDB/kubectl-cnp/raw/main/install.sh
| sh -s -- -b .

# Login to the OpenShift Cluster

oc login ...

# Create your namespace

oc new-project userXX
```


Checking operator installation

Scripts

03_check_operator_installed.sh

Checking operator installation

In Kubernetes, the operator is by default installed in the postgresql-operator-system namespace as a Kubernetes Deployment. The name of this deployment depends on the installation method. When installed through the manifest or the cnp plugin, by default, it is called postgresql-operator-controller-manager. When installed via Helm, by default, the deployment name is derived from the helm release name, appended with the suffix -edb-postgres-for-kubernetes (e.g., <name>-edb-postgres-for-kubernetes).

Doc [link](#)

Code

```
# Check operator installation

oc get deploy -A | grep postgres
NAMESPACE:    openshift-operators
NAME:         postgresql-operator-controller-manager
READY:        1/1
UP-TO-DATE:   1
AVAILABLE:    1
AGE:          5d21h
```



Install Postgres cluster

Scripts

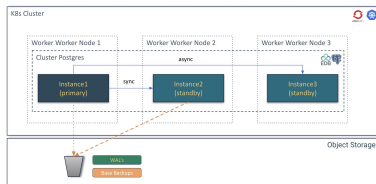
```
04-prepare-cluster.sh
05_install_cluster.sh
06_show_status.sh
07_insert_data.sh
```

Prepare Postgres cluster template

Setup credentials (AWS or Minio) and prepare context for your users

Deploy Postgres cluster in HA

Different types of architectures can be deployed. In our demo, 3 instances are deployed with HA available.



Insert Data

Insert data in test table

Doc [link](#)

Code

```
export cluster_name="cluster-userXX"
export TMP=$TMPDIR

# Get template
envsubst < templates/${cluster_name}-template.yaml >
$TMP/${cluster_name}.yaml

# Deploy Postgres cluster
${kubectl_cmd} apply -f $TMP/${cluster_name}.yaml

-----
ApiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: cluster-example
spec:
  instances: 3
  imageName: "docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6"
  enableSuperuserAccess: true

  replicationSlots:
    highAvailability:
      enabled: true
```



Monitoring

Show status

Cluster Summary

Name: default/cluster-sergio-1
System ID: 7566239054312464401
PostgreSQL Image: docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6
Primary instance: cluster-sergio-1
Primary promotion time: 2025-10-28 11:52:15 +0000 UTC (4m30s)
Status: Cluster in healthy state
Instances: 3
Ready instances: 3
Size: 127M
Current Write LSN: 0/6032300 (Timeline: 1 - WAL File: 00000001000000000000000006)

Continuous Backup status (Barman Cloud Plugin)

ObjectStore / Server name: object-store/cluster-sergio
First Point of Recoverability: -
Last Successful Backup: -
Last Failed Backup: -
Working WAL archiving: OK
WALs waiting to be archived: 0
Last Archived WAL: 000000010000000000000005.0000028.backup @ 2025-10-28T11:54:11.908195Z
Last Failed WAL: -

Streaming Replication status

Replication Slots Enabled											
Name	Sent LSN	Write LSN	Flush LSN	Replay LSN	Write Lag	Flush Lag	Replay Lag	State	Sync State	Sync Priority	Replication Slot
cluster-sergio-2	0/9000000	0/9000000	0/9000000	0/9000000	00:00:00	00:00:00	00:00:00	streaming	quorum	1	active
cluster-sergio-3	0/9000000	0/9000000	0/9000000	0/9000000	00:00:00	00:00:00	00:00:00	streaming	quorum	1	active

Instances status

Name	Current LSN	Replication role	Status	QoS	Manager Version	Node
cluster-sergio-1	0/9000000	Primary	OK	Guaranteed	1.28.0	ocp4-multiarch-worker1-ppc64
cluster-sergio-2	0/9000000	Standby (sync)	OK	Guaranteed	1.28.0	ocp4-multiarch-w5gg4-master-2
cluster-sergio-3	0/9000000	Standby (sync)	OK	Guaranteed	1.28.0	ocp4-multiarch-w5gg4-master-1



Promote and Failover

Scripts

08_promote.sh

Promote

The meaning of this command is to promote a pod in the cluster to primary, so you can start with maintenance work or test a switch-over situation in your cluster:

Doc [link](#)

Code

```
# Promote
. ./config.sh

${kubectl_cnp} promote ${cluster_name} ${cluster-name}-2
```

Name of Cluster
Eg: cluster-userxx

Name of the 2nd pod
Eg: cluster-userxx-2



Minor upgrade

Scripts

09_upgrade.sh

Minor upgrade

To upgrade to a newer minor version, simply update the PostgreSQL container image reference in your cluster definition, either directly or via image catalogs. CloudNativePG will trigger a rolling update of the cluster, replacing each instance one by one, starting with the replicas. Once all replicas have been updated, it will perform either a switchover or a restart of the primary to complete the process.

Doc [link](#)

Code

```
# Minor upgrade

# From
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: cluster-sergio1
spec:
  instances: 3
  imageName:
    "docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6"
  ...

# To
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: cluster-sergio1
spec:
  instances: 3
  imageName:
    "docker.enterprisedb.com/k8s/edb-postgres-advanced:17.7"
  ...
```



Backups

Scripts

```
10_backup_cluster.sh  
11_backup_describe.sh
```

Backup

Backups are used using Barman cloud with the new Barman Plugin. In case the plugin is not compatible (like in Red Hat OpenShift), we will continue using Barman Cloud (without the plugin).

PostgreSQL natively provides first class backup and recovery capabilities based on file system level (physical) copy. These have been successfully used for more than 15 years in mission critical production databases, helping organizations all over the world achieve their disaster recovery goals with Postgres.

Doc [link](#)

Code

```
cat $TMP/backup.yaml  
  
apiVersion: postgresql.k8s.enterprisedb.io/v1  
kind: Backup  
metadata:  
  name: cluster-example-backup-test  
spec:  
  cluster:  
    name: cluster-example  
  
${kubect1_cmd} apply -f ./yaml/backup.yaml
```



Recovery

Scripts

12_restore_cluster.sh

Recovery

In PostgreSQL, recovery refers to the process of starting an instance from an existing physical backup. PostgreSQL's recovery system is robust and feature-rich, supporting Point-In-Time Recovery (PITR)—the ability to restore a cluster to any specific moment, from the earliest available backup to the latest archived WAL file.

Doc [link](#)

Code

```
cat $TMP/restore.yaml

apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: cluster-restore
spec:
  instances: 1
  imageName: quay.io/enterprisedb/postgresql:16.4
  imagePullPolicy: IfNotPresent
  bootstrap:
    recovery:
      source: source
  plugins:
  - name: barman-cloud.cloudnative-pg.io
    isWALArchiver: true
    parameters:
      barmanObjectName: object-store
  externalClusters:
  - name: source
    plugin:
      name: barman-cloud.cloudnative-pg.io
      parameters:
        barmanObjectName: object-store
        serverName: cluster-example

${kubectl_cmd} apply -f $TMP/restore.yaml
```



Promote and Failover

Scripts

13_failover.sh

Automated Failover

In the case of unexpected errors on the primary for longer than the `.spec.failoverDelay` (by default 0 seconds), the cluster will go into failover mode. This may happen, for example, when:

- The primary pod has a disk failure
- The primary pod is deleted
- The postgres container on the primary has any kind of sustained failure

In the failover scenario, the primary cannot be assumed to be working properly.
Doc [link](#)

Code

```
# Automatic Failover (simulate failover killing pods of primary)
${kubect1_cmd} delete pvc/${primary} \
                      pvc/${primary}-wal \
                      pod/${primary} \
                      --force
```



Scale out, scale down

Scripts

```
14_scale_out.sh  
15_scale_down.sh
```

Scale out and down

The operator allows you to scale up and down the number of instances in a PostgreSQL cluster. New replicas are started up from the primary server and participate in the cluster's HA infrastructure.

Doc [link](#)

Code

```
# Scale out  
kubectl scale cluster cluster-example --replicas=4  
  
# Scale down  
kubectl scale cluster cluster-example --replicas=2
```



Fencing and Hibernation

Scripts

```
30_fencing_on.sh  
31_fencing_off.sh
```

Fencing

Fencing is the process of protecting the data in one, more, or even all instances of a PostgreSQL cluster when they appear to be malfunctioning. When an instance is fenced, the PostgreSQL server process is guaranteed to be shut down, while the pod is kept running. This ensures that, until the fence is lifted, data on the pod isn't modified by PostgreSQL and that you can investigate file system for debugging and troubleshooting purposes.

Doc [link](#)

Code

```
# Fencing on  
${kubectl_cnp} fencing on ${cluster_name} ${replica}  
  
# Fencing off  
${kubectl_cnp} fencing off ${cluster_name} ${replica}
```



Fencing and Hibernation

Scripts

32_hibernation_on.sh
33_hibernation_off.sh

Hibernation

CloudNativePG supports hibernation of a running PostgreSQL cluster in a declarative manner, through the `cnpg.io/hibernation` annotation. Hibernation enables saving CPU power by removing the database pods while keeping the database PVCs. This feature simulates scaling to 0 instances.

Doc [link](#)

Code

```
# Hibernation on
${kubectl_cmd} annotate cluster ${cluster_name} \
    --overwrite k8s.enterprisedb.io/hibernation=on

# Hibernation off
${kubectl_cmd} annotate cluster ${cluster_name} \
    --overwrite k8s.enterprisedb.io/hibernation=off
```



Major upgrade (copying data)

Scripts

20_upgrade_major_version.sh

Major upgrade (with native logical replication)

Major PostgreSQL releases introduce changes to the internal data storage format, requiring a more structured upgrade process.

CloudNativePG supports three methods for performing major upgrades:

- Logical dump/restore – Blue/green deployment, offline.
- Native logical replication – Blue/green deployment, online.
- **Physical with pg_upgrade** – In-place upgrade, offline (covered in the "Offline In-Place Major Upgrades" section below).

Doc [link](#)

Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: ${cluster_major_upgrade}
spec:
  instances: 1
  imageName: ${postgres_major_upgrade_image}

  storage:
    size: 2Gi

  bootstrap:
    initdb:
      import:
        type: microservice
        databases:
          - app
        source:
          externalCluster: ${cluster_name}

  externalClusters:
    - name: ${cluster_name}
      connectionParameters:
        # Use the correct IP or host name for the source database
        host: ${cluster_name}-rw
        user: postgres
        dbname: postgres
      password:
        name: ${cluster_name}-superuser
        key: password
```



Major upgrade (copying data)

Scripts

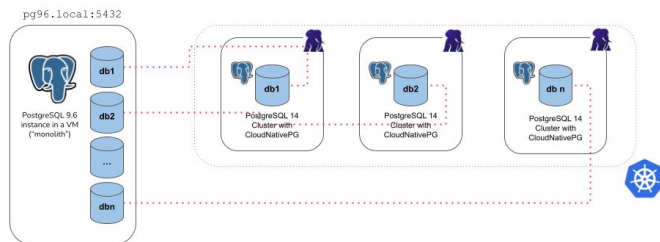
20_upgrade_major_version.sh

Major upgrade (with native logical replication)

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting **"import"** in the bootstrap section
- The operator uses internally postgres tools `pg_dump` and `pg_restore`
- This method can be used to **migrate** another database or to run

out-of-the-place upgrade

- Possible settings:
- Microservices
- Monolith



Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: cluster-sergio-major
spec:
  instances: 1
  imageName: docker.enterprisedb.com/k8s/postgresql:18.1
  imagePullSecrets:
    - name: postgresql-operator-pull-secret
  enableSuperuserAccess: true

  bootstrap:
    initdb:
      import:
        type: microservice
        databases:
          - postgres
        source:
          externalCluster: cluster-sergio

  externalClusters:
    - name: cluster-sergio
      connectionParameters:
        host: cluster-sergio-rw
        user: postgres
        dbname: postgres
      password:
        name: cluster-sergio-superuser
        key: password
    ...
```

Already finished ?



Application deployment

Database : CNPG Potsgres cluster
Backend : Rest API - Postgrest
Frontend : Swagger UI
Frontend : World Cup UI

Update manifest postgrest-secret.yaml

Get the URI of your instance

```
oc get secret cluster-<NAMESPACE>-app -o jsonpath='{.data.uri}' |  
base64 -d  
Replace in PGRST_DB_URI
```

Update manifest postgrest-deployment.yaml

Adapt the fqdn

```
PGRST_OPENAPI_SERVER_PROXY_URI =  
http://postgrest-<namespace>.apps-crc.testing
```

```
PGRST_SERVER_CORS_ALLOWED_ORIGINS  
http://swagger-ui-<namespace>.apps-crc.testing,http://worldcup-ui-<name  
space>.apps-crc.testing
```

Update the frontend to point to your backend

Deploy `./packaging/deploy.sh`

Code

▼ edb-postgres-for-kubernetes-in-openshift

- ▼ app
 - > database
 - ▼ packaging
 - > db
 - ▼ k8s
 - ! db-dump-configmap.yaml U
 - ! db-restore-job.yaml U
 - ! frontend.yaml U
 - ! postgrest-config.yaml U
 - ! postgrest-deployment.yaml U
 - ! postgrest-secret.yaml U
 - ! postgrest-service-route.yaml U
 - ! swagger-ui.yaml U
 - \$ deploy.sh U
 - i README.md U
 - \$ undeploy.sh U
 - > worldcup-ui
 - i README.md U



Thank you

For any queries, contact:

sergio.romera@enterprisedb.com



Backup Slides



Architectures



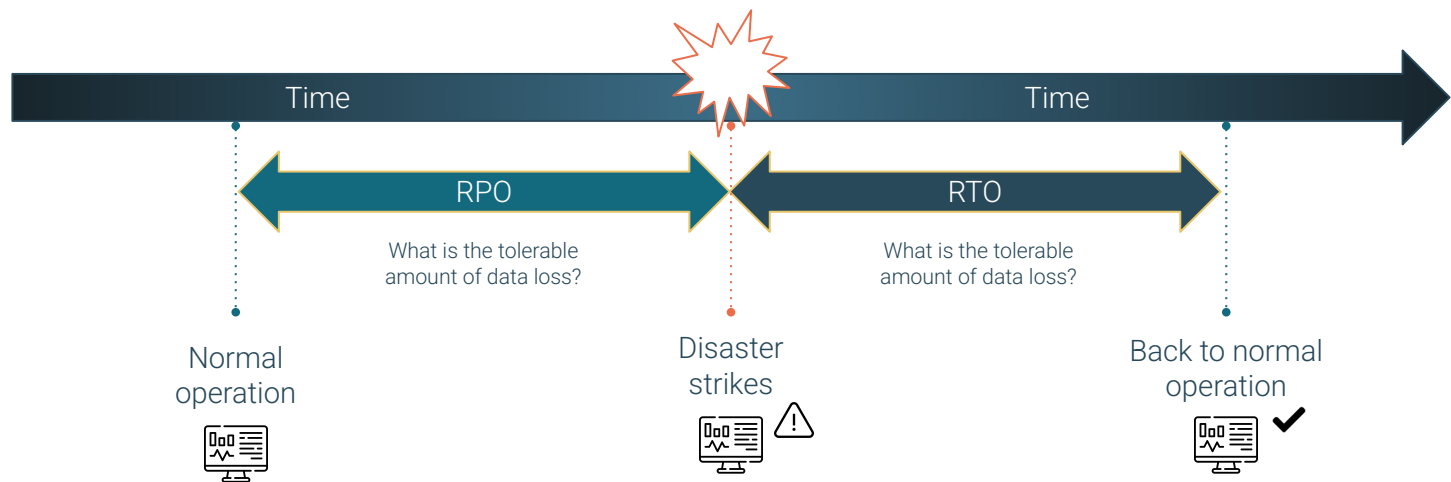
When to choose Kubernetes over VMs?

- 01 |** Cloud Native Applications that already run in Kubernetes
- 02 |** Scalable, replicated databases
- 03 |** Applications requiring automated failover and self-healing
- 04 |** Teams skilled in Kubernetes who want a unified infrastructure



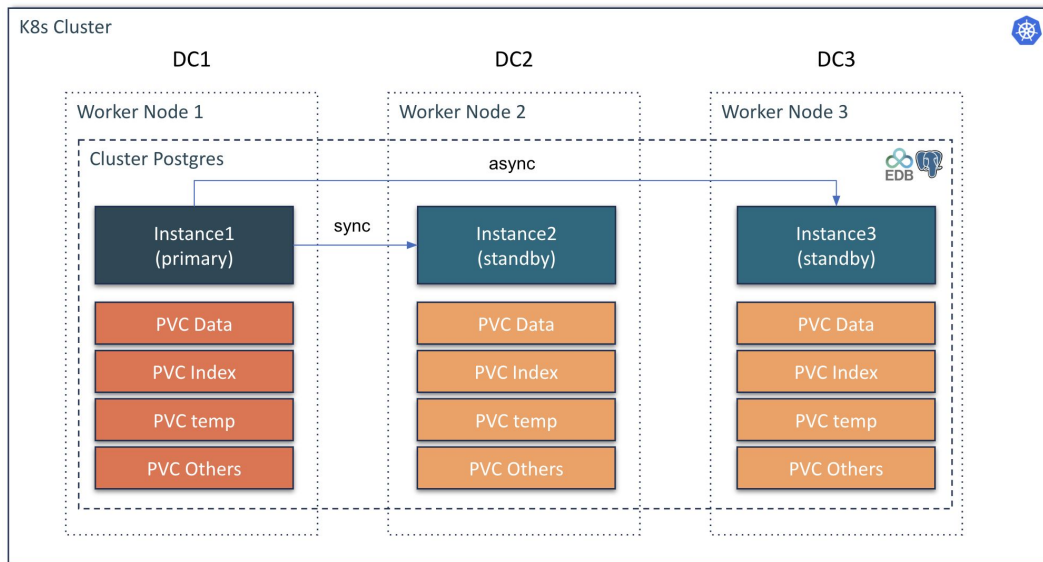
Concepts

- Recovery Point Objective (**RPO**) and Recovery Time Objective (**RTO**) are key concepts in disaster recovery and business continuity planning, particularly related to data loss and system downtime.



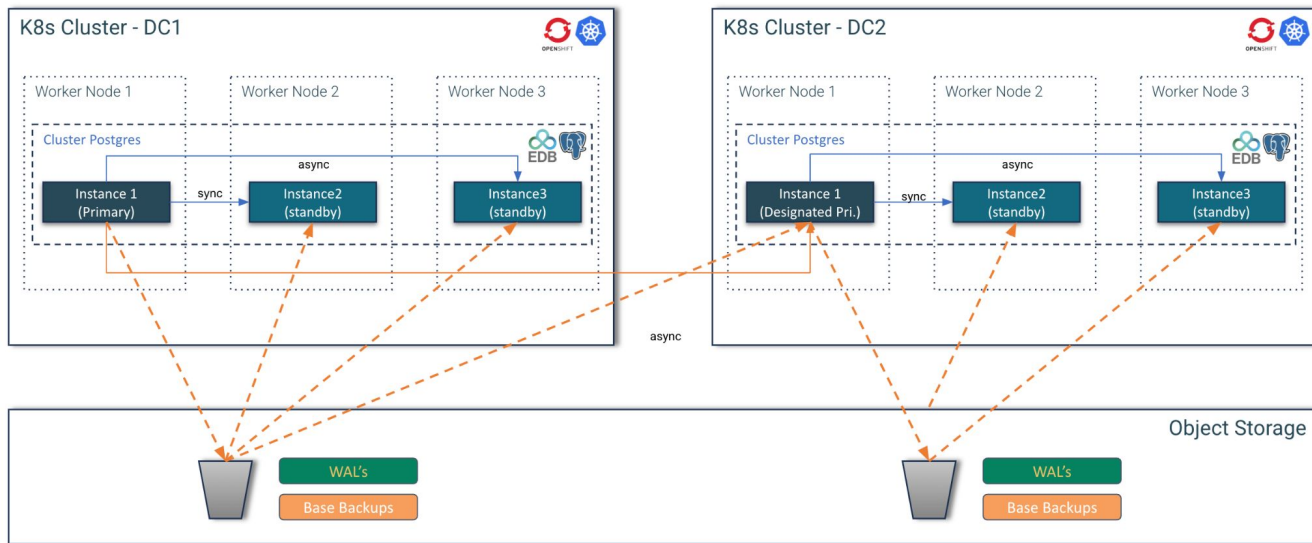
Red Hat Recommendation

Red Hat recommend stretched clusters ONLY when latencies don't exceed 5 milliseconds (ms) round-trip time (RTT) between the nodes in different locations, with a maximum RTT of 10 ms.

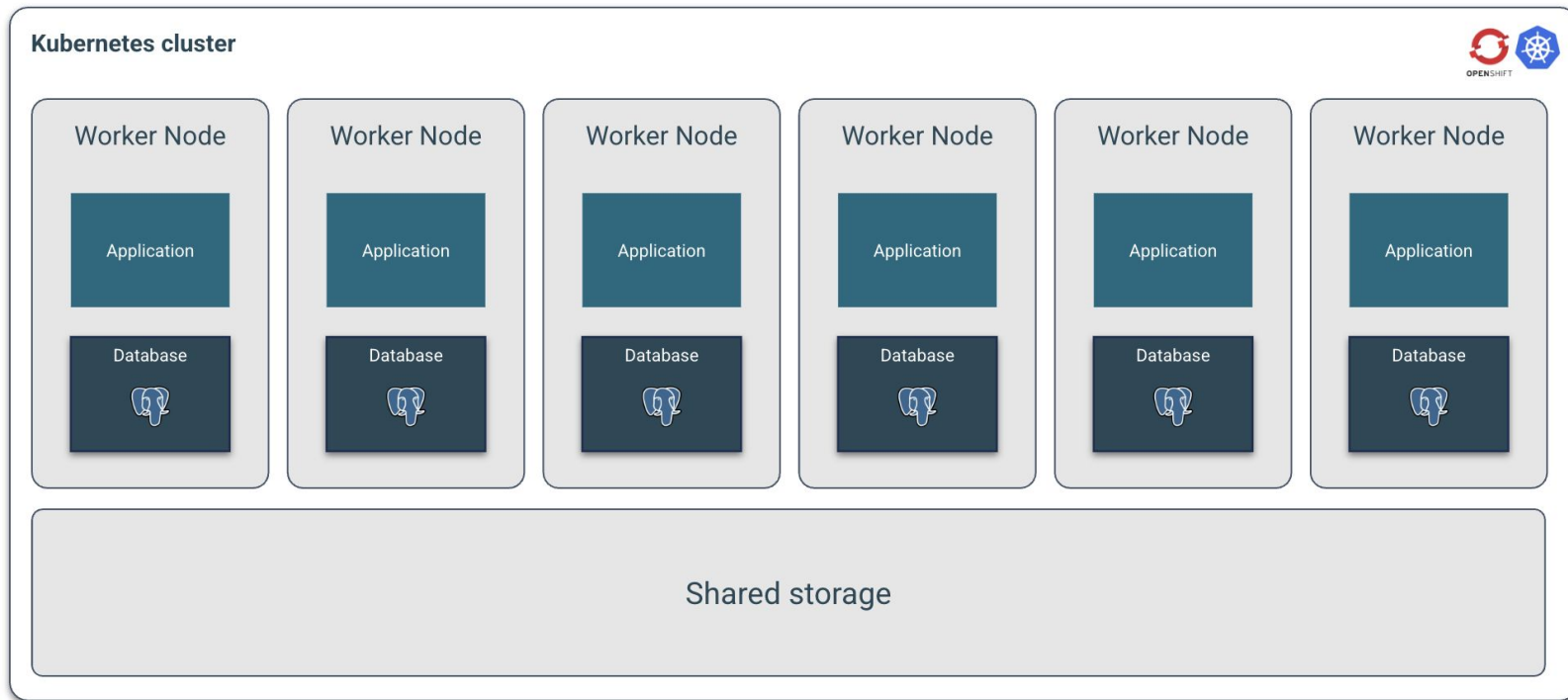


Two separate single data center Kubernetes clusters

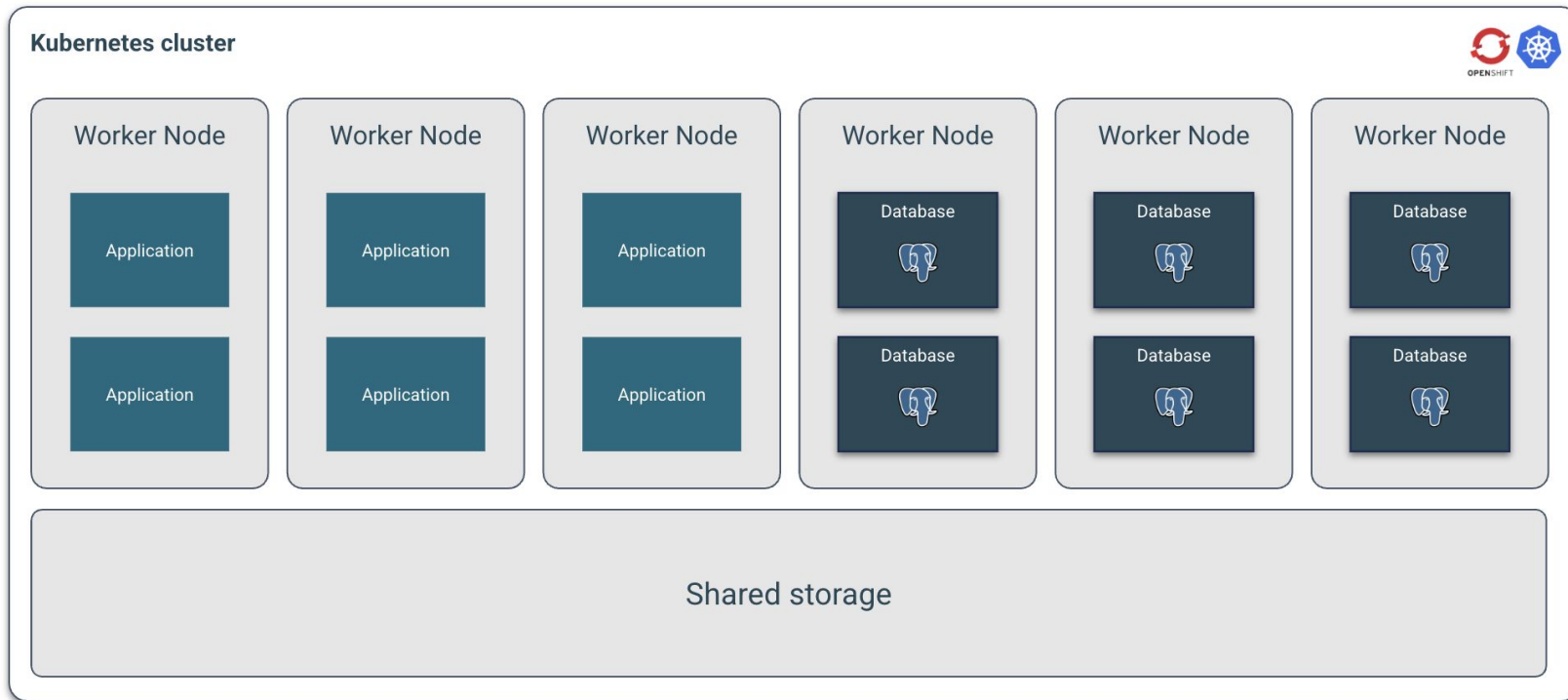
In case you cannot go beyond two data centers and you end up with two separate Kubernetes clusters, don't despair.



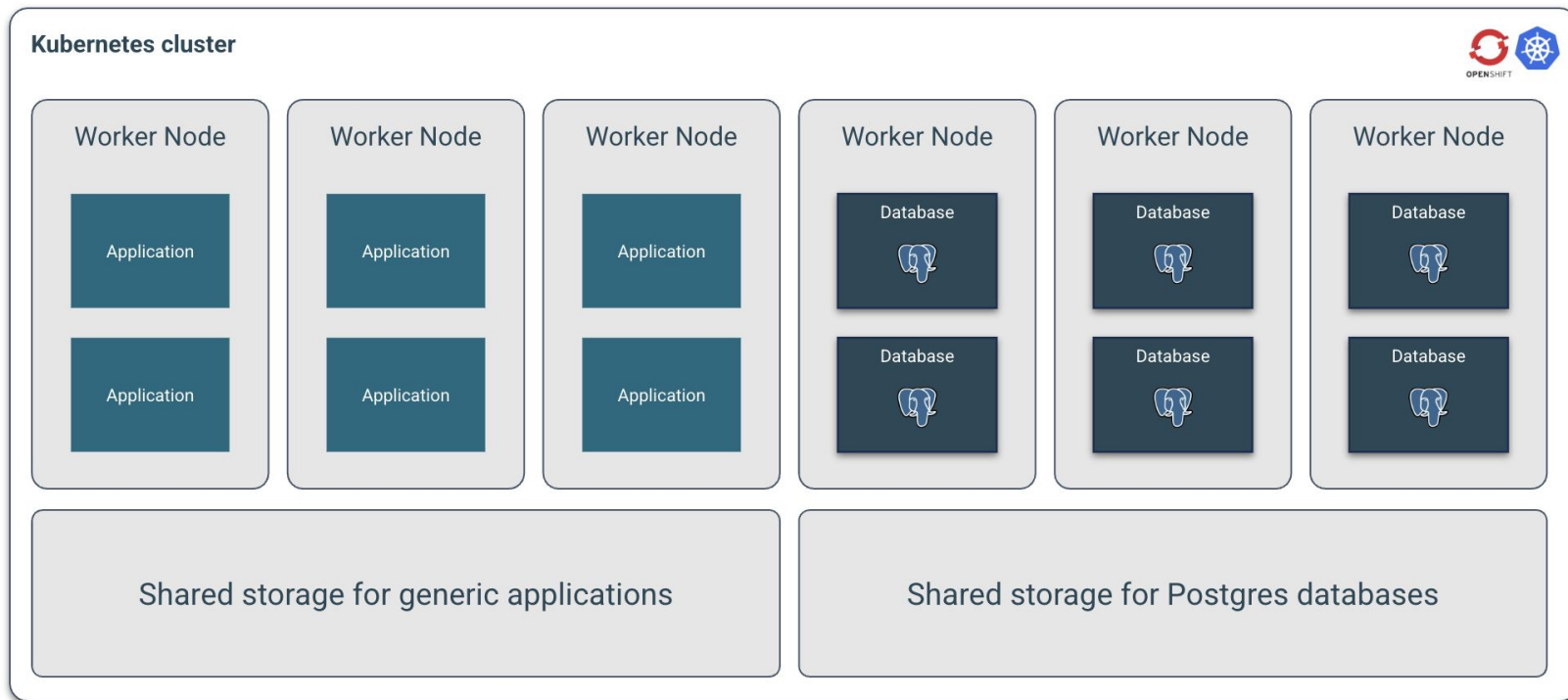
Shared workload, shared storage



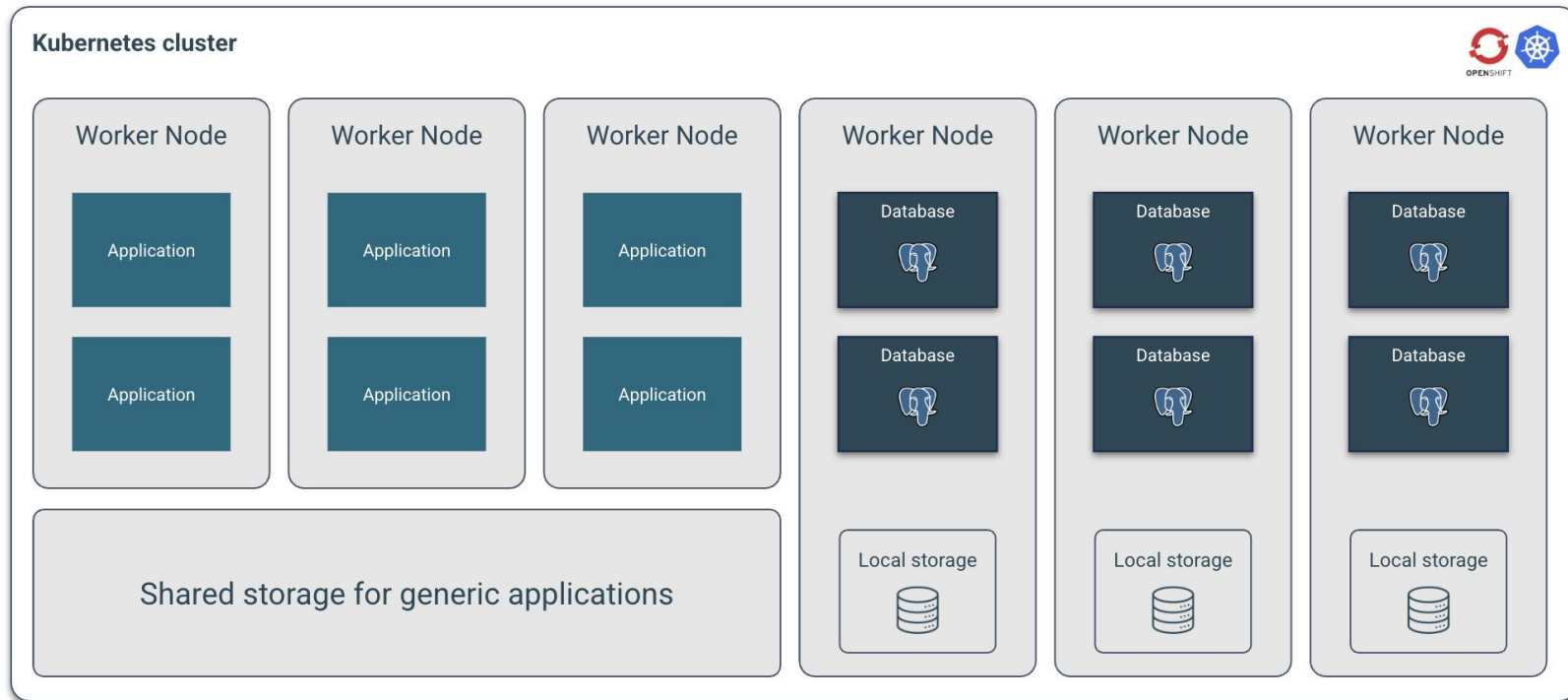
Shared workload, shared storage



Shared workload, shared storage

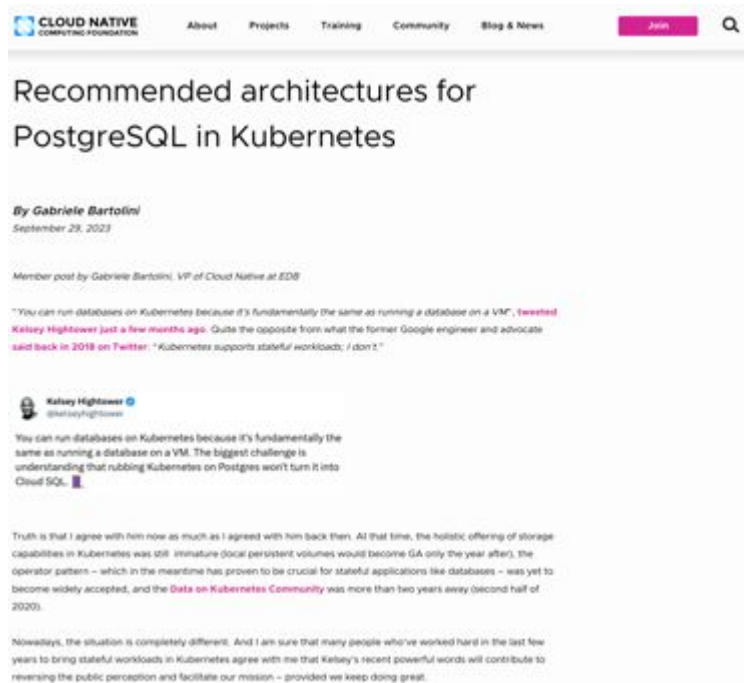


Shared workloads, local storage



Recommended architectures

<https://www.cncf.io/blog/2023/09/29/recommended-architectures-for-postgresql-in-kubernetes/>

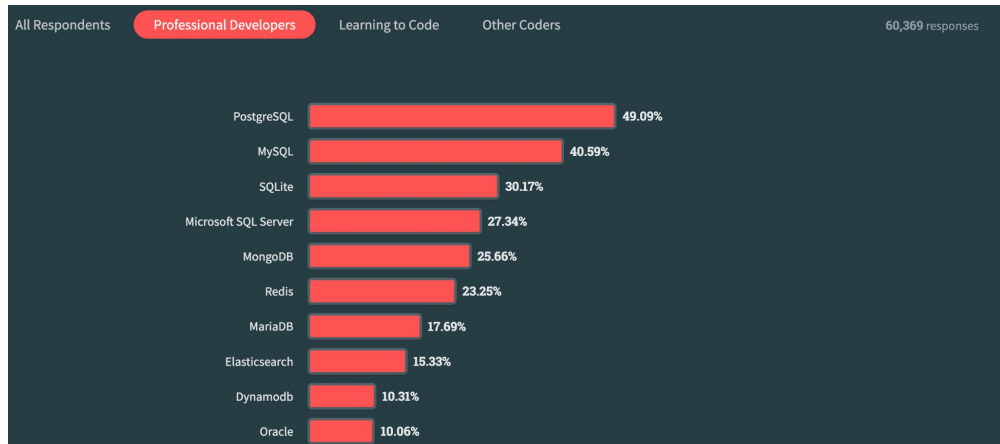


PostgreSQL most loved database (2023)

Source: [Stackoverflow](#)

This year, PostgreSQL took over the first place spot from MySQL. Professional Developers are more likely than those learning to code to use PostgreSQL (50%) and those learning are more likely to use MySQL (54%).

MongoDB is used by a similar percentage of both Professional Developers and those learning to code and it's the second most popular database for those learning to code (behind MySQL).



Links:

Openshift Console:

<https://console-openshift-console.apps.cluster-826x2.826x2.sandbox5464.opentlc.com/>

Users:

name: user2..user32
Password: edb-workshop

Devspaces url:

<https://devspaces.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>

Short url to Devspaces:

<https://tinyurl.com/mrxs95pb>

Minio:

UI: <https://minio-ui-minio.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>
API: <https://minio-api-minio.apps.cluster-826x2.826x2.sandbox5464.opentlc.com>

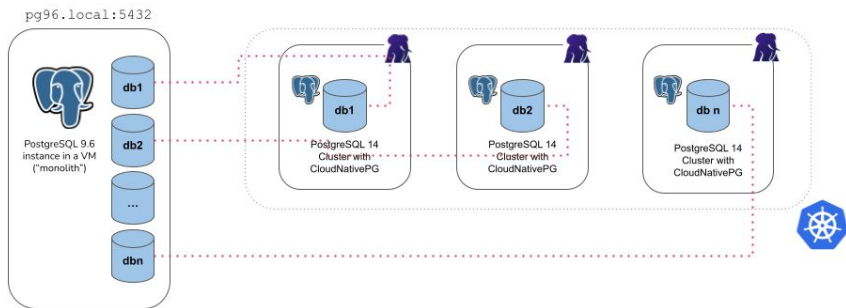
User: minio
Password: edb-workshop



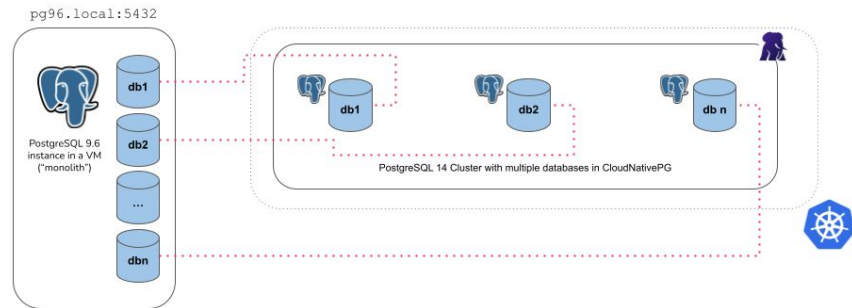
Database Migration

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting “**import**” in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:

Microservices:



Monolith:



Major upgrade (copying data)

Scripts

```
24_major_upgrade_in_place.sh
25_verify_major_upgrade_16_17.sh
```

Major upgrade (offline In-Place Major Upgrades)

CloudNativePG performs an offline in-place major upgrade when a new operand container image with a higher PostgreSQL major version is declaratively requested for a cluster.

Major upgrades are only supported between images based on the same operating system distribution. For example, if your previous version uses a bullseye image, you cannot upgrade to a bookworm image.

Doc [link](#)

Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
  name: ${cluster_major_upgrade}
spec:
  instances: 1
  imageName: ${postgres_major_upgrade_image}

  storage:
    size: 2Gi

  bootstrap:
    initdb:
      import:
        type: microservice
        databases:
          - app
  ...
```



Disaster recovery

Scripts

```
./dr/01-create-all.sh  
./dr/02-insert-eu.sh  
./dr/03-switchover-cluster.sh
```

Disaster recovery

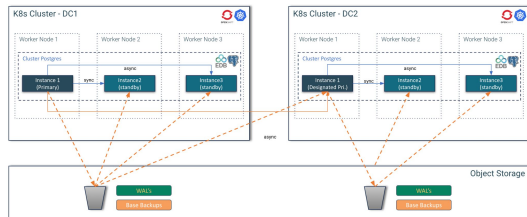
A replica cluster is a CloudNativePG Cluster resource designed to replicate data from another PostgreSQL instance, ideally also managed by CloudNativePG.

Typically, a replica cluster is deployed in a different Kubernetes cluster in another region. These clusters can be configured to perform cascading replication and can rely on object stores for data replication from the source, as detailed further down..

Two clusters:

- pg-eu
- pg-us

Doc [link](#)



Code

```
# EU  
kubectl delete objectstores.barmancloud.cnpg.io object-storage-eu  
envsubst < ../templates/dr/azure-object-storage-eu.yaml >  
./tmp/object-storage-eu.yaml  
kubectl apply -f ./tmp/object-storage-eu.yaml  
sleep 2  
rm -f ./tmp/pg-eu.yaml  
envsubst < ../templates/dr/pg-eu.yaml > ./tmp/pg-eu.yaml  
print info "Creating pg-eu cluster...\n"  
kubectl apply -f ./tmp/pg-eu.yaml  
print_info "pg-eu cluster created.\n"  
  
# US  
kubectl delete objectstores.barmancloud.cnpg.io object-storage-us  
envsubst < ../templates/dr/azure-object-storage-us.yaml >  
./tmp/object-storage-us.yaml  
kubectl apply -f ./tmp/object-storage-us.yaml  
sleep 2  
rm -f ./tmp/pg-us.yaml  
envsubst < ../templates/dr/pg-us.yaml > ./tmp/pg-us.yaml  
kubectl wait --timeout=30m --for=condition=Ready cluster/pg-eu  
print info "Creating pg-us cluster...\n"  
kubectl apply -f ./tmp/pg-us.yaml  
./backup_cluster.sh pg-eu  
print_info "pg-us cluster created.\n"
```


The “cnp” plugin for kubectl

- The official CLI for CloudNativePG
 - Available also as RPM or Deb package
- Extends the ‘kubectl’ command:
 - Customize the installation of the operator
 - Status of a cluster
 - Perform a manual switchover (promote a standby) or a restart of a node
 - Issue TLS certificates for client authentication
 - Declare start and stop of a Kubernetes node maintenance
 - Destroy a cluster and all its PVC
 - Fence a cluster or a set of the instances
 - Hibernate a cluster
 - Generate jobs for benchmarking via pgbench and fio
 - Issue a new backup
 - Start pgadmin



Name: cluster-example
Namespace: default
System ID: 7100921006673293335
PostgreSQL Image: ghcr.io/cloudnative-pg/postgresql:14.3
Primary instance: cluster-example-2
Status: Cluster in healthy state
Instances: 3
Ready instances: 3
Current Write LSN: 0/C000060 (Timeline: 4 - WAL File: 00000004000000000000000C)

Certificates Status

Certificate Name	Expiration Date	Days Left Until Expiration
cluster-example-replication	2022-08-21 13:15:00 +0000 UTC	89.95
cluster-example-server	2022-08-21 13:15:00 +0000 UTC	89.95
cluster-example-ca	2022-08-21 13:15:00 +0000 UTC	89.95

Continuous Backup status

First Point of Recoverability: 2022-05-23T13:37:08Z
Working WAL archiving: OK
WALs waiting to be archived: 0
Last Archived WAL: 00000004000000000000000B @ 2022-05-23T13:42:09.37537Z
Last Failed WAL: -

Streaming Replication status

Name	Sent LSN	Write LSN	Flush LSN	Replay LSN	Write Lag	Flush Lag	Replay Lag	State	Sync State	Sync Priority
cluster-example-3	0/C000060	0/C000060	0/C000060	0/C000060	00:00:00	00:00:00	00:00:00	streaming	async	0
cluster-example-1	0/C000060	0/C000060	0/C000060	0/C000060	00:00:00	00:00:00	00:00:00	streaming	async	0

Instances status

Name	Database Size	Current LSN	Replication role	Status	QoS	Manager Version
cluster-example-3	33 MB	0/C000060	Standby (async)	OK	BestEffort	1.15.0
cluster-example-2	33 MB	0/C000060	Primary	OK	BestEffort	1.15.0
cluster-example-1	33 MB	0/C000060	Standby (async)	OK	BestEffort	1.15.0



Install CNPG plugin

NOT NEEDED DURING WORKSHOP
For illustrative purposes.

- In the web terminal run the script 01_install_plugin.sh:
./01_install_plugin.sh



Call to action: Check CNPG plugin

- Call the help for the CNPG Plugin, run:
`kubectl-cnp help`



Use case

Operator installation



Operator Installation demonstration

NOT NEEDED DURING WORKSHOP
For illustrative purposes.

- Install the operator
- Check the installed CNP Operator in the console
- Discover the features of the Operator in the OpenShift environment
- Check the installed CNP Operator in the web terminal



Install the CNPG Operator and check the it in the web terminal

NOT NEEDED DURING WORKSHOP
For illustrative purposes.

- In the web terminal install the operator:

```
./02_install_operator.sh
```

-> will require admin privs on Openshift



Call to action: Check the it in the web terminal

- In the web terminal check the installation of the operator:

```
./03_check_operator_installed.sh
```



Call to action: Check the installed CNPG Operator in the Openshift console

- In the OpenShift console navigate to:
 - -> Operators
 - -> Installed Operators
 - -> Klick on the Operator installed:

The screenshot shows the Red Hat OpenShift console interface. On the left is a sidebar with navigation links: Home, Favorites, Operators (expanded), OperatorHub, Installed Operators (selected), Helm, Workloads, Networking, Storage, Builds, and Observe. The main content area is titled 'Installed Operators' and shows a table of installed operators. A green arrow points to the 'EDB Postgres for Kubernetes' operator.

Name	Namespace	Managed Namespaces	Status	Last updated	Provided APIs
 cert-manager Operator for Red Hat OpenShift 1.17.0 provided by Red Hat	 cert-manager-operator	 cert-manager-operator	✓ Succeeded Up to date	14. Okt. 2025, 12:19	CertificateRequest Certificate Challenge ClusterIssuer View 4 more...
 EDB Postgres for Kubernetes 1.27.0 provided by EDB	 openshift-operators	All Namespaces	✓ Succeeded Up to date	15. Okt. 2025, 05:43	Backups Cluster Image Catalog Cluster Postgres Database View 6 more...

Call to action: Discover the features of the Operator in the OpenShift environment

Project: openshift-operators ▾

[Installed Operators](#) > Operator details



EDB Postgres for Kubernetes

1.27.0 provided by EDB



Details

[YAML](#)

[Subscription](#)

[Events](#)

[All instances](#)

[Backups](#)

[Cluster Image Catalog](#)

[Cluster](#)

[Postgres Database](#)

[Failover Qu](#)

Provided APIs

Backups

PostgreSQL backup (physical base backup)

[+ Create instance](#)

Cluster Image Catalog

A cluster-wide catalog of PostgreSQL operand images

[+ Create instance](#)

Cluster

PostgreSQL cluster (primary/standby architecture)

[+ Create instance](#)

Postgres Database

Declarative creation and management of a database on a Cluster

[+ Create instance](#)

Failover Quorum

FailoverQuorum contains the information about the current failover quorum status of a PG cluster

[+ Create instance](#)

Image Catalog

A catalog of PostgreSQL operand images

[+ Create instance](#)

Pooler

Postgres Publication

Scheduled Backups



Use case

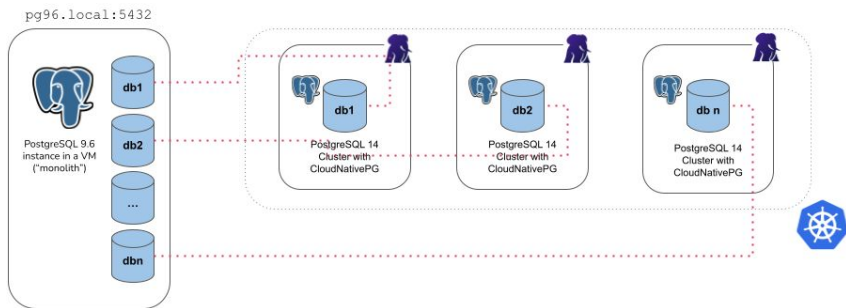
Database Migration / Major Version Upgrade



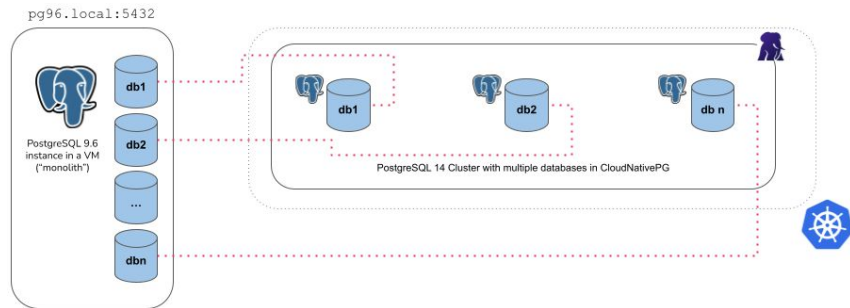
Database Migration

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting “**import**” in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:

Microservices:



Monolith:



Call to action: Run migration or out of the place upgrade

- In the web terminal 1:

- Run the script:

- ```
./20_upgrade_major_version.sh
```

- Check the cluster creation process:

- ```
kubectl get pods -w
```

- Check the table test in the cluster-user<X>-17, run the command:

- Connect to the cluster:

- ```
oc cnp psql cluster-user<X>-17
```

- Connect to the database app:

- ```
\c app
```

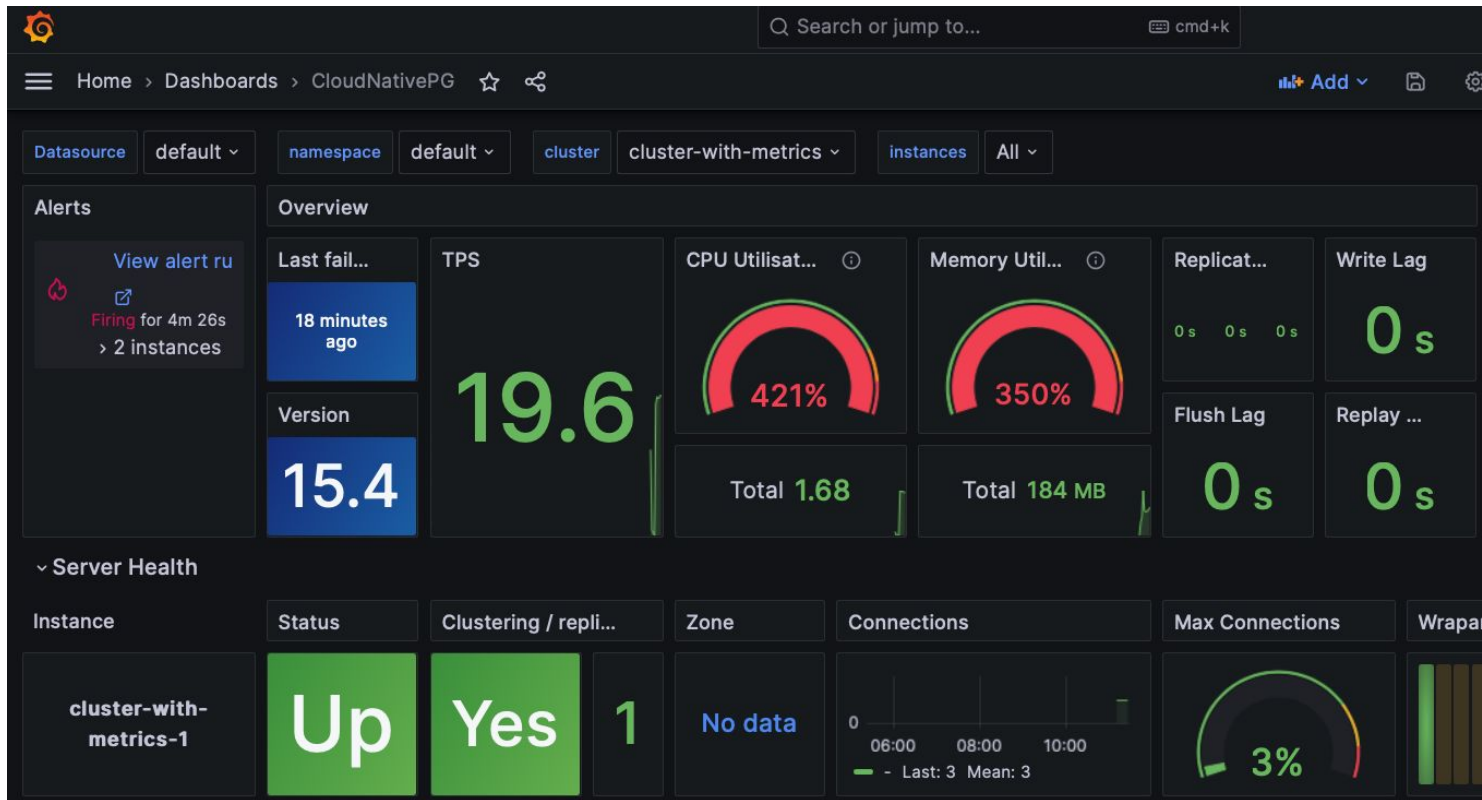
- Run sql commands:

- ```
select version();
```

- ```
select count(*) from test;
```



Grafana Dashboard



Agenda

Start	End	Session
12:30	13:30	Registration & Welcome Lunch
13:30	13:45	Camptocamp Vorstellung
13:45	14:00	Red Hat OpenShift & EDB Partnerschaft
14:00	14:15	Einführung in den Postgres-Marketplace
14:15	14:30	Coffee
14:30	15:00	CNPG Operator Reference Architecture
15:00	15:30	Functionalities for CNPG
15:30	17:15	Interactive session & demo
17:15	18:30	Drinks & Aperero

